

TP Jour 2 — Logs E-Commerce avec HDFS

TP Jour 2 — Traitement de Logs E-Commerce : Introduction à HDFS

Formation : Data Engineering — Apache Airflow **École** : IPSSI Montpellier **Niveau** : M2 **Durée estimée** : 3h00 **Thème** : Ingestion et analyse de logs Apache dans HDFS via Airflow

Contexte professionnel

Vous êtes data engineer au sein de l'équipe **Data Platform** d'une marketplace française de type Cdiscount. La plateforme e-commerce génère environ **50 Go de logs Apache par jour** répartis sur une dizaine de serveurs web frontaux.

Ces logs contiennent des informations critiques pour les équipes :

- **Équipe Ops** : détection des pics d'erreurs 5xx (incidents serveur), analyse des timeouts
- **Équipe Produit** : top des pages visitées, taux de rebond, navigation utilisateur
- **Équipe Sécurité** : détection d'IPs suspectes, tentatives de scraping, anomalies de trafic

Aujourd'hui, les logs sont purgés toutes les 48h sur les serveurs web faute d'espace disque, ce qui rend toute analyse rétrospective impossible. L'objectif de ce TP est de construire un **pipeline d'ingestion quotidienne** vers **HDFS** (Hadoop Distributed File System), qui constitue la couche de stockage brut de votre architecture Lambda.

Technologies utilisées :

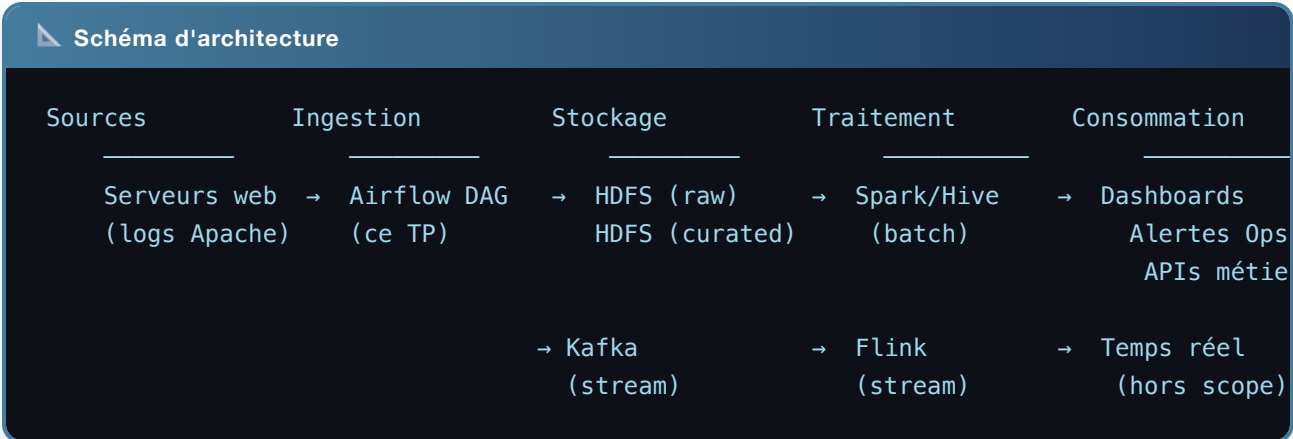
Outil	Version	Rôle
Apache Airflow	2.8	Orchestration du pipeline

Outil	Version	Rôle
Apache Hadoop (HDFS)	3.2.1	Stockage distribué des logs
Docker Compose	v2	Déploiement local du stack
BranchPythonOperator	—	Routage conditionnel selon le taux d'erreur
HdfsSensor	—	Attente de disponibilité du fichier dans HDFS

Partie 1 — Architecture Lambda et place de HDFS

1.1 Architecture Lambda de la marketplace

L'architecture Lambda distingue trois couches de traitement :



1.2 Zones HDFS

HDFS est organisé en zones fonctionnelles qui correspondent aux étapes de transformation des données :

Schéma d'architecture

```
/data/ecommerce/
├── logs/
│   ├── raw/                Zone atterrissage brute – fichiers tels quels depuis les
│   │   ├── 2024/03/15/
│   │   │   └── access_2024-03-15.log
│   │   └── processed/      Zone après traitement – logs validés et analysés
│   │       ├── 2024/03/15/
│   │       │   └── access_2024-03-15.log
│   └── reports/            Rapports JSON générés par les analyses
└── tmp/                   Fichiers temporaires (nettoyés périodiquement)
```

Règle d'or : On ne supprime jamais les données de la zone `raw`. En cas de bug dans un traitement, on peut toujours rejouer le pipeline depuis les données brutes.

1.3 Architecture NameNode / DataNode

HDFS suit une architecture maître/esclave :

Schéma d'architecture



Dans ce TP, vous n'aurez qu'un **seul DataNode** (configuration minimale locale). En production, le facteur de réplication est typiquement 3.

Partie 2 — Setup Docker : Airflow + Hadoop

2.1 Structure des répertoires

▸ Schéma d'architecture

```
ecommerce-logs-pipeline/
├── docker-compose.yaml
├── hadoop.env
├── .env
├── dags/
│   └── logs_ecommerce_dag.py
├── logs/
├── plugins/
└── scripts/
    └── generer_logs.py
```

\$ Bash

```
mkdir -p ecommerce-logs-pipeline/{dags,logs,plugins,scripts}
cd ecommerce-logs-pipeline
```

2.2 Fichier `hadoop.env`

Ce fichier centralise la configuration Hadoop partagée entre NameNode et DataNode :

\$ Bash

```
CORE_CONF_fs_defaultFS=hdfs://namenode:9000
CORE_CONF_hadoop_http_staticuser_user=root
CORE_CONF_hadoop_proxyuser_hue_hosts=*
CORE_CONF_hadoop_proxyuser_hue_groups=*
CORE_CONF_io_compression_codecs=org.apache.hadoop.io.compress.SnappyCodec

HDFS_CONF_dfs_webhdfs_enabled=true
HDFS_CONF_dfs_permissions_enabled=false
HDFS_CONF_dfs_namenode_datanode_registration_ip__hostname__check=false
HDFS_CONF_dfs_replication=1
```

2.3 Fichier `docker-compose.yaml`

📁 Schéma d'architecture

```
version: '3.8'

# — Configuration Airflow partagée —
x-airflow-common: &airflow-common
  image: apache/airflow:2.8.0
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@pos
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__API__AUTH_BACKENDS: 'airflow.api.auth.backend.basic_auth,airflow.api.
    _PIP_ADDITIONAL_REQUIREMENTS: 'requests apache-airflow-providers-apache-hdfs'
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./plugins:/opt/airflow/plugins
    - ./scripts:/opt/airflow/scripts
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on:
    postgres:
      condition: service_healthy

services:

# — Base de données Airflow —
postgres:
  image: postgres:13
  environment:
    POSTGRES_USER: airflow
    POSTGRES_PASSWORD: airflow
    POSTGRES_DB: airflow
  volumes:
    - postgres-db-volume:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "airflow"]
    interval: 10s
    retries: 5
    start_period: 5s
    restart: always

# — Airflow —
airflow-webserver:
  <<: *airflow-common
  command: webserver
  ports:
```

```

    - "8080:8080"
healthcheck:
  test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
  interval: 30s
  timeout: 10s
  retries: 5
  start_period: 30s
  restart: always

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  restart: always

airflow-init:
  <<: *airflow-common
  entrypoint: /bin/bash
  command:
    - -c
    - |
      airflow db init &&
      airflow users create \
        --username admin --firstname Admin --lastname User \
        --role Admin --email admin@example.com --password admin
  environment:
    <<: *airflow-common
    _AIRFLOW_DB_UPGRADE: 'true'

# — Hadoop HDFS —
namenode:
  image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
  container_name: namenode
  restart: always
  ports:
    - "9870:9870" # Interface Web HDFS
    - "9000:9000" # Port RPC HDFS (communication client)
  volumes:
    - hadoop_namenode:/hadoop/dfs/name
  env_file:
    - hadoop.env
  environment:
    CLUSTER_NAME: ecommerce-cluster

datanode:
  image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
  container_name: datanode
  restart: always
  volumes:
    - hadoop_datanode:/hadoop/dfs/data
  env_file:
    - hadoop.env

```

```
environment:
  SERVICE_PRECONDITION: "namenode:9870"
depends_on:
  - namenode

volumes:
  postgres-db-volume:
  hadoop_namenode:
  hadoop_datanode:
```

2.4 Démarrage et vérification

▴ Schéma d'architecture

```
# Fichier .env pour Airflow
echo "AIRFLOW_UID=50000" > .env

# Initialiser Airflow
docker compose up airflow-init

# Démarrer tout le stack
docker compose up -d

# Vérifier l'état des conteneurs
docker compose ps
```

Vérification HDFS :

▴ Schéma d'architecture

```
# Rapport de santé du cluster HDFS
docker exec namenode hdfs dfsadmin -report

# Liste des fichiers à la racine HDFS
docker exec namenode hdfs dfs -ls /

# Créer les répertoires de travail
docker exec namenode hdfs dfs -mkdir -p /data/ecommerce/logs/raw
docker exec namenode hdfs dfs -mkdir -p /data/ecommerce/logs/processed

# Vérifier la création
docker exec namenode hdfs dfs -ls /data/ecommerce/logs/
```

Interfaces web disponibles :

Service	URL	Identifiants
Airflow UI	http://localhost:8080	admin / admin
HDFS NameNode Web UI	http://localhost:9870	—

Partie 3 — Connexion Airflow vers HDFS

Pour qu’Airflow puisse interagir avec HDFS via le provider officiel, vous devez créer une connexion dans l’interface.

1. Ouvrez Airflow UI → **Admin** → **Connections**
2. Cliquez sur + (Ajouter une connexion)
3. Remplissez les champs suivants :

Champ	Valeur
Connection Id	hdfs_default
Connection Type	HDFS
Host	namenode
Port	9870
Login	root

4. Cliquez **Save**

Note : Dans ce TP, nous utiliserons principalement des `BashOperator` avec des commandes `docker exec namenode hdfs dfs ...`, ce qui évite les problèmes de configuration réseau entre conteneurs Airflow et HDFS. L’`HdfsSensor` en revanche utilisera la connexion `hdfs_default`.

Partie 4 — Script de génération de logs

Créez le fichier `scripts/generer_logs.py`. Ce script génère des logs Apache au format **Combined Log Format** réalistes, similaires à ce que produirait une vraie plateforme e-commerce.


```
#!/usr/bin/env python3
"""
Générateur de logs Apache Combined Log Format pour simulation e-commerce.
Usage : python3 generer_logs.py <date> <nb_lignes> <fichier_sortie>
Exemple : python3 generer_logs.py 2024-03-15 1000 /tmp/access_2024-03-15.log
"""

import random
import sys
from datetime import datetime, timedelta

# --- Données réalistes d'une marketplace ---

IPS = [
    "92.184.12.44", "185.220.101.12", "78.23.145.67", "213.95.11.88",
    "5.188.10.132", "91.108.4.15", "176.31.208.51", "62.210.114.199",
    "37.187.0.200", "82.66.14.25", "90.54.12.144", "51.15.201.77",
    "109.190.122.6", "212.47.234.67", "163.172.0.100",
]

URLS = [
    # Pages produits (trafic élevé)
    ("GET", "/produit/smartphone-samsung-galaxy-s24-ultra", 200, 48200),
    ("GET", "/produit/laptop-dell-inspiron-15-amd", 200, 72100),
    ("GET", "/produit/casque-audio-sony-wh1000xm5", 200, 38900),
    ("GET", "/produit/aspirateur-dyson-v15", 200, 41000),
    ("GET", "/produit/montre-connectee-apple-watch-9", 200, 55300),
    # Pages catégories
    ("GET", "/categorie/informatique", 200, 22400),
    ("GET", "/categorie/electromenager", 200, 19800),
    ("GET", "/categorie/smartphones", 200, 25100),
    # Pages panier / checkout (critique)
    ("GET", "/panier", 200, 8900),
    ("POST", "/checkout/valider", 200, 3400),
    ("POST", "/checkout/paiement", 200, 4100),
    # Erreurs courantes
    ("GET", "/produit/article-retire-de-vente", 404, 512),
    ("GET", "/admin/dashboard", 403, 287),
    ("GET", "/produit/inexistant-xyz", 404, 512),
    ("POST", "/api/panier/ajouter", 500, 1024),
    ("GET", "/api/stock/verifier", 503, 890),
    # Assets statiques
    ("GET", "/static/css/main.min.css", 200, 18200),
    ("GET", "/static/js/bundle.min.js", 200, 234100),
    ("GET", "/static/img/logo.png", 200, 4200),
    ("GET", "/favicon.ico", 200, 1150),
]
```

```

USER_AGENTS = [
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/121.0 Saf
    'Mozilla/5.0 (Macintosh; Intel Mac OS X 14_3) AppleWebKit/605.1.15 Safari/605.
    'Mozilla/5.0 (iPhone; CPU iPhone OS 17_3) AppleWebKit/605.1.15 Mobile/15E148',
    'Mozilla/5.0 (Linux; Android 14; Pixel 8) AppleWebKit/537.36 Chrome/121.0 Mobi
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:122.0) Gecko/20100101 Firefox/12
    'python-requests/2.31.0', # Bot / scraper
    'Googlebot/2.1 (+http://www.google.com/bot.html)',
]

REFERRERS = [
    'https://www.google.fr/search?q=smartphone+pas+cher',
    'https://www.google.fr/shopping',
    'https://www.bing.com/search?q=laptop+dell',
    'https://www.lesnumeriques.com/',
    '-', # Direct
    '-',
    '-',
]

def generer_log_line(date_str: str) -> str:
    """Génère une ligne de log Apache Combined Log Format."""
    ip = random.choice(IPS)
    methode, url, status, taille = random.choices(
        URLs,
        weights=[u[2] for u in URLs], # Pondération par status code (200 plus fré
        k=1
    )[0]
    user_agent = random.choice(USER_AGENTS)
    referrer = random.choice(REFERRERS)

    # Horodatage réaliste dans la journée
    base = datetime.strptime(date_str, "%Y-%m-%d")
    delta = timedelta(
        hours=random.randint(0, 23),
        minutes=random.randint(0, 59),
        seconds=random.randint(0, 59),
    )
    timestamp = (base + delta).strftime("%d/%b/%Y:%H:%M:%S +0100")

    # Format Combined Log Format : IP - - [timestamp] "METHOD URL HTTP/1.1" status
    return (
        f'{ip} - - [{timestamp}] "{methode} {url} HTTP/1.1" '
        f'{status} {taille} "{referrer}" "{user_agent}"'
    )

def main():
    if len(sys.argv) != 4:
        print("Usage: python3 generer_logs.py <date YYYY-MM-DD> <nb_lignes> <fichi

```

```
sys.exit(1)

date_str = sys.argv[1]
nb_lignes = int(sys.argv[2])
fichier_sortie = sys.argv[3]

with open(fichier_sortie, "w") as f:
    for _ in range(nb_lignes):
        f.write(generer_log_line(date_str) + "\n")

print(f"[OK] {nb_lignes} lignes générées dans {fichier_sortie}")

if __name__ == "__main__":
    main()
```

Test rapide en local :

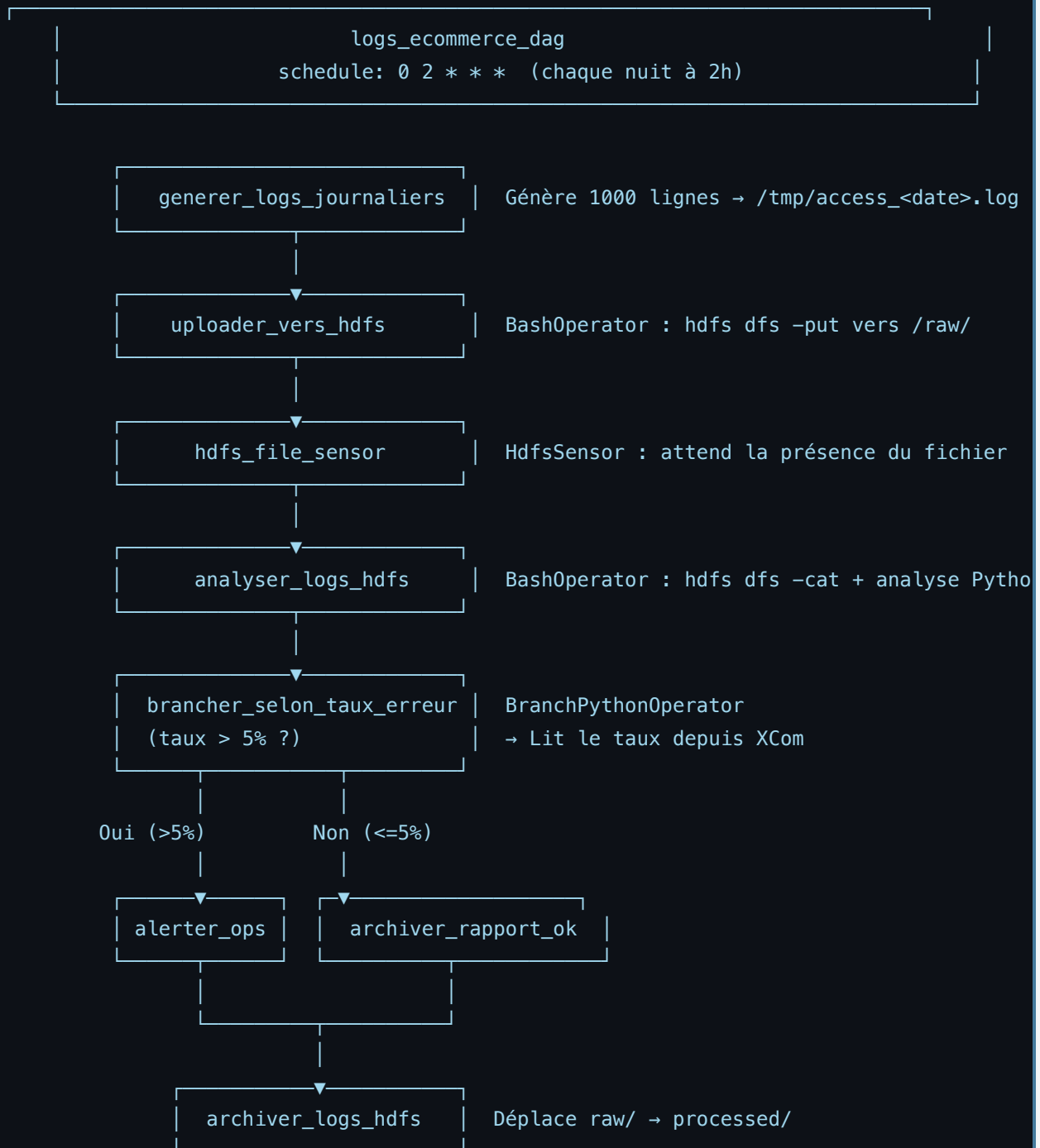
\$ Bash

```
python3 scripts/generer_logs.py 2024-03-15 100 /tmp/test_access.log
head -5 /tmp/test_access.log
```

Partie 5 — Architecture du DAG

Le DAG `logs_ecommerce_dag.py` comporte **8 tâches** avec un branchement conditionnel basé sur le taux d'erreur détecté :

Schéma d'architecture



Partie 6 — Travail demandé

Étape 1 — Démarrer le stack Docker

1. Créez tous les fichiers de configuration (Parties 2 et 3).

2. Lancez `docker compose up airflow-init` puis `docker compose up -d`.
3. Attendez que tous les conteneurs soient `healthy` ou `running`.
4. Vérifiez HDFS : `docker exec namenode hdfs dfsadmin -report` doit afficher 1 DataNode `Live datanodes`.
5. Accédez à la Web UI HDFS : <http://localhost:9870> — explorez l'onglet **Datanodes** et **Browse Directory**.

Étape 2 — Créer les répertoires HDFS

Créez la structure HDFS décrite en Partie 1.2 :

Schéma d'architecture

```
# Créer les répertoires de travail
docker exec namenode hdfs dfs -mkdir -p /data/ecommerce/logs/raw
docker exec namenode hdfs dfs -mkdir -p /data/ecommerce/logs/processed

# Vérifier
docker exec namenode hdfs dfs -ls -R /data/

# Ajuster les permissions
docker exec namenode hdfs dfs -chmod -R 777 /data/
```

Vérifiez également depuis la Web UI HDFS (**Browse Directory** → `/data/ecommerce/logs/`).

Étape 3 — Implémenter `generer_logs_journaliers()`

▸ Schéma d'architecture

```
import subprocess
import logging
from datetime import date

def generer_logs_journaliers(**context):
    """
    Génère 1000 lignes de logs Apache pour la date d'exécution du DAG.
    Sauvegarde dans /tmp/access_<YYYY-MM-DD>.log.
    Retourne le chemin du fichier (stocké dans XCom pour les tâches suivantes).
    """
    # La date logique d'exécution (pas forcément aujourd'hui si catchup=True)
    execution_date = context["ds"] # Format YYYY-MM-DD
    fichier_sortie = f"/tmp/access_{execution_date}.log"
    script_path = "/opt/airflow/scripts/generer_logs.py"

    # À compléter :
    # 1. Utiliser subprocess.run() pour appeler le script Python :
    #    ["python3", script_path, execution_date, "1000", fichier_sortie]
    # 2. Passer check=True pour lever une exception si le script échoue
    # 3. Vérifier que le fichier existe et logger sa taille en octets
    # 4. Retourner fichier_sortie

    pass
```

Point important : Utilisez `context["ds"]` (data stamp) plutôt que `date.today()`. Cela garantit l'idempotence : si Airflow rejoue la tâche pour le 10 mars, les logs générés seront bien ceux du 10 mars, pas ceux d'aujourd'hui.

Étape 4 — Implémenter `uploader_vers_hdfs()`

Cette tâche utilise un `BashOperator` pour copier le fichier local vers HDFS. Notez que le fichier est sur le système de fichiers du **conteneur Airflow scheduler**, et que HDFS est accessible depuis le **conteneur namenode**.

Schéma d'architecture

```
from airflow.operators.bash import BashOperator

# Dans la définition du DAG :

t_upload = BashOperator(
    task_id="uploader_vers_hdfs",
    bash_command="""
        EXECUTION_DATE="{{ ds }}"
        FICHER_LOCAL="/tmp/access_{{EXECUTION_DATE}}.log"
        CHEMIN_HDFS="/data/ecommerce/logs/raw/access_{{EXECUTION_DATE}}.log"

        echo "[INFO] Upload de {{FICHER_LOCAL}} vers HDFS:{{CHEMIN_HDFS}}"

        # Copie du fichier depuis le conteneur airflow-scheduler
        # vers le conteneur namenode (via volume partagé ou commande docker)
        # À compléter : utiliser la commande hdfs dfs -put appropriée
        # Indice : docker exec namenode hdfs dfs -put <source> <destination>
        # Attention : le fichier /tmp/ n'est pas directement accessible depuis nam
        # Solution : d'abord copier via docker cp, puis hdfs dfs -put depuis namen

        echo "[OK] Upload terminé"
    """,
)
```

Réflexion : Pourquoi ne peut-on pas faire directement `docker exec namenode hdfs dfs -put /tmp/access_*.log ...` ? Quel est le problème avec le montage de volumes entre les conteneurs airflow-scheduler et namenode dans ce docker-compose ? Proposez et implémentez une solution.

Étape 5 — Implémenter le `HdfsSensor`

Le `HdfsSensor` bloque l'exécution du pipeline jusqu'à ce que le fichier soit détecté dans HDFS. C'est un mécanisme de sécurité pour ne pas analyser un fichier qui ne serait pas encore complètement écrit.

Schéma d'architecture

```
from airflow.providers.apache.hdfs.sensors.hdfs import HdfsSensor

# Dans la définition du DAG :

t_sensor = HdfsSensor(
    task_id="hdfs_file_sensor",
    filepath="/data/ecommerce/logs/raw/access_{{ ds }}.log",
    hdfs_conn_id="hdfs_default",
    poke_interval=30,          # Vérifie toutes les 30 secondes
    timeout=300,              # Abandonne après 5 minutes
    mode="poke",              # Mode synchrone (bloque un slot de worker)
)
```

Note : Si le `HdfsSensor` ne fonctionne pas avec votre configuration réseau Docker, vous pouvez le remplacer par un `BashOperator` qui exécute `docker exec namenode hdfs dfs -test -e /data/ecommerce/logs/raw/access_{{ ds }}.log && echo "OK" || exit 1`.

Étape 6 — Implémenter `analyser_logs_hdfs()`

▢ Schéma d'architecture

```
from airflow.operators.bash import BashOperator
import re
import logging

# Approche : BashOperator qui lit le fichier HDFS et l'analyse via Python inline

t_analyser = BashOperator(
    task_id="analyser_logs_hdfs",
    bash_command="""
        EXECUTION_DATE="{{ ds }}"
        CHEMIN_HDFS="/data/ecommerce/logs/raw/access_${EXECUTION_DATE}.log"

        echo "[INFO] Lecture du fichier HDFS : ${CHEMIN_HDFS}"

        # Lire le fichier depuis HDFS
        docker exec namenode hdfs dfs -cat "${CHEMIN_HDFS}" > /tmp/logs_analyse_${EXECUTION_DATE}.txt

        echo "[INFO] Analyse des logs..."

        # Compter les requêtes par status code
        echo "=== STATUS CODES ==="
        grep -oP '"[A-Z]+ [^ ]+ HTTP/[0-9.]+" [0-9]+' /tmp/logs_analyse_${EXECUTION_DATE}.txt \
            | grep -oP '[0-9]+$' | sort | uniq -c | sort -rn

        # Top 5 URLs les plus visitées
        echo "=== TOP 5 URLS ==="
        grep -oP '"(GET|POST) [^ ]+' /tmp/logs_analyse_${EXECUTION_DATE}.txt \
            | cut -d' ' -f2 | sort | uniq -c | sort -rn | head -5

        # Calculer le taux d'erreur
        TOTAL=$(wc -l < /tmp/logs_analyse_${EXECUTION_DATE}.txt)
        ERREURS=$(grep -cP '"[A-Z]+ [^ ]+ HTTP/[0-9.]+" (4|5)[0-9]{2}' \
            /tmp/logs_analyse_${EXECUTION_DATE}.txt || echo 0)
        echo "=== TAUX ERREUR ==="
        echo "Total: ${TOTAL}, Erreurs: ${ERREURS}"

        # Sauvegarder le taux pour la tâche suivante (via fichier temporaire)
        echo "${ERREURS} ${TOTAL}" > /tmp/taux_erreur_${EXECUTION_DATE}.txt
    """,
)
```

Travail demandé : Complétez si nécessaire et faites fonctionner cette tâche. Vérifiez les logs dans l'UI Airflow.

Étape 7 — BranchPythonOperator

Le `BranchPythonOperator` décide de la branche à emprunter selon le taux d'erreur calculé à l'étape précédente.

```

from airflow.operators.python import BranchPythonOperator

SEUIL_ERREUR_PCT = 5.0 # Seuil d'alerte : 5% d'erreurs HTTP

def brancher_selon_taux_erreur(**context):
    """
    Lit le taux d'erreur calculé par analyser_logs_hdfs.
    Retourne le task_id de la branche à exécuter.
    """
    execution_date = context["ds"]
    fichier_taux = f"/tmp/taux_erreur_{execution_date}.txt"

    # À compléter :
    # 1. Lire le fichier /tmp/taux_erreur_{date}.txt
    # 2. Parser les valeurs ERREURS et TOTAL
    # 3. Calculer le taux : taux_pct = (erreurs / total) * 100
    # 4. Logger le taux
    # 5. Retourner "alerter_equipe_ops" si taux > SEUIL_ERREUR_PCT
    #    sinon retourner "archiver_rapport_ok"
    pass

def alerter_equipe_ops(**context):
    """Simule l'envoi d'une alerte à l'équipe Ops (Slack, PagerDuty, etc.)."""
    execution_date = context["ds"]
    logging.warning(
        f"[ALERTE] Taux d'erreur HTTP anormal détecté pour les logs du {execution_date}. Vérifiez les serveurs web."
    )
    # En production : appel API Slack, PagerDuty, ou envoi email via SMTP

def archiver_rapport_ok(**context):
    """Logue que tout est nominal et archive un rapport de bonne santé."""
    execution_date = context["ds"]
    logging.info(
        f"[OK] Taux d'erreur dans les seuils normaux pour les logs du {execution_date}"
    )

# Dans la définition du DAG :

t_branch = BranchPythonOperator(
    task_id="brancher_selon_taux_erreur",
    python_callable=brancher_selon_taux_erreur,
)

t_alerte = PythonOperator(

```

```

        task_id="alerter_equipe_ops",
        python_callable=alerter_equipe_ops,
    )

    t_archive_ok = PythonOperator(
        task_id="archiver_rapport_ok",
        python_callable=archiver_rapport_ok,
    )

```

Point clé : La tâche finale `archiver_logs_hdfs` doit s'exécuter quelle que soit la branche choisie. Utilisez `trigger_rule="none_failed_min_one_success"` sur cette tâche.

Étape 8 — Archiver les logs dans HDFS

La tâche finale déplace les logs de la zone `raw` vers la zone `processed` pour signifier que le traitement est terminé.

▢ Schéma d'architecture

```

t_archiver = BashOperator(
    task_id="archiver_logs_hdfs",
    bash_command="""
        EXECUTION_DATE="{{ ds }}"
        SOURCE="/data/ecommerce/logs/raw/access_${EXECUTION_DATE}.log"
        DESTINATION="/data/ecommerce/logs/processed/access_${EXECUTION_DATE}.log"

        echo "[INFO] Déplacement HDFS : ${SOURCE} → ${DESTINATION}"

        # À compléter :
        # Utiliser hdfs dfs -mv pour déplacer le fichier
        # Commande : docker exec namenode hdfs dfs -mv <source> <destination>

        echo "[OK] Fichier archivé dans la zone processed"
    """,
    trigger_rule="none_failed_min_one_success",
)

```

Dépendances complètes du DAG :

 Python

```
(  
    t_generer  
    >> t_upload  
    >> t_sensor  
    >> t_analyser  
    >> t_branch  
    >> [t_alerte, t_archive_ok]  
    >> t_archiver  
)
```

Partie 7 — Questions de réflexion

Répondez dans un fichier `REPONSES.md`.

Q1 — HDFS vs système de fichiers local Pourquoi ne pas simplement stocker les logs sur le disque local du serveur Airflow ou sur un NFS ? Listez 3 avantages concrets de HDFS pour un cas d'usage de 50 Go/jour de logs, en vous appuyant sur les caractéristiques du système (distribution, réplication, localité des données).

Q2 — NameNode, point de défaillance unique (SPOF) Dans l'architecture HDFS, le NameNode est un SPOF. Si le NameNode tombe, que se passe-t-il pour les DataNodes et pour les clients ? Quels mécanismes Hadoop propose-t-il pour pallier ce problème en production (HDFS NameNode HA) ? Quel est le rôle du Journal Node dans cette architecture haute disponibilité ?

Q3 — HdfsSensor vs polling actif Comparez le `HdfsSensor` en mode `poke` et en mode `reschedule`. Dans quel cas utiliseriez-vous l'un plutôt que l'autre ? Quel est l'impact sur le nombre de slots de workers Airflow disponibles ? Proposez un scénario concret où le mauvais choix de mode bloquerait tout le scheduler.

Q4 — Réplication HDFS et cohérence des données Dans le `hadoop.env`, on a `HDFS_CONF_dfs_replication=1` (réplication minimale). En production avec un facteur de réplication de 3, expliquez ce qui se passe lors de l'écriture d'un bloc de 128 Mo : combien de copies sont écrites, sur combien de DataNodes, et dans quel ordre ? Que garantit HDFS en termes de cohérence lors d'une lecture concurrente (pendant que l'écriture est en cours) ?

Livrables

À déposer sur le portail de la formation avant la fin de séance :

Schéma d'architecture

```
ecommerce-logs-pipeline/
├── docker-compose.yaml      (fourni, peut être modifié si justifié)
├── hadoop.env
├── .env
├── dags/
│   └── logs_ecommerce_dag.py  DAG complet et fonctionnel
├── scripts/
│   └── generer_logs.py        Script de génération de logs
└── REPONSES.md               Réponses aux 4 questions + captures d'écran
```

Captures d'écran à inclure dans `REPONSES.md` :

1. `docker compose ps` montrant tous les conteneurs en état `running` ou `healthy`
2. `docker exec namenode hdfs dfsadmin -report` montrant 1 DataNode Live
3. Interface Web HDFS (<http://localhost:9870>) — onglet Browse Directory avec `/data/ecommerce/logs/raw/`
4. Airflow UI — Vue Graph du DAG `logs_ecommerce_dag` avec les 8 tâches
5. Airflow UI — Exécution complète avec les 2 branches visibles (une verte, une grisée)
6. Logs de la tâche `analyser_logs_hdfs` montrant le résumé des status codes et le Top 5 URLs
7. Interface Web HDFS — fichier déplacé dans `/data/ecommerce/logs/processed/` après exécution

TP conçu pour la formation Data Engineering — IPSSI Montpellier — Apache Airflow 2.8 + Hadoop 3.2.1

Exercices Supplémentaires

Ces 3 exercices approfondissent HDFS, les Sensors et la résilience des pipelines.

Exercice 1 — HdfsSensor : attendre la disponibilité d'un fichier (☆☆)

Contexte : En architecture Lambda, plusieurs pipelines peuvent déposer des fichiers dans HDFS indépendamment. Votre pipeline d'analyse ne doit démarrer que lorsque le fichier de logs est effectivement présent dans HDFS.

Objectif : Remplacer la dépendance directe `stocker_hdfs → analyser_logs` par un **Sensor** qui attend activement la présence du fichier.

CONCEPT SENSOR DANS AIRFLOW

Un Sensor est une tâche spéciale qui se ré-exécute en boucle jusqu'à ce qu'une condition soit vraie :

📄 Schéma d'architecture

```
from airflow.sensors.base import BaseSensorOperator

class HdfsFileSensor(BaseSensorOperator):
    """Sensor qui attend qu'un fichier existe dans HDFS."""

    def __init__(self, hdfs_path: str, namenode_url: str, **kwargs):
        super().__init__(**kwargs)
        self.hdfs_path = hdfs_path
        self.namenode_url = namenode_url

    def poke(self, context) -> bool:
        """
        Méthode appelée à chaque itération du sensor.
        Retourne True si la condition est satisfaite (sensor terminé).
        Retourne False pour continuer à attendre.
        """
        import requests
        try:
            resp = requests.get(
                f"{self.namenode_url}/webhdfs/v1{self.hdfs_path}",
                params={"op": "GETFILESTATUS", "user.name": "root"},
                timeout=5,
            )
            if resp.status_code == 200:
                size = resp.json()["FileStatus"]["length"]
                self.log.info("Fichier trouvé : %s (%d bytes)", self.hdfs_path, size)
                return True
        except Exception as exc:
            self.log.warning("Fichier absent ou HDFS KO : %s", exc)
        return False
```

TRAVAIL DEMANDÉ

1. Implémentez `HdfsFileSensor` dans un fichier `plugins/hdfs_sensor.py`.
2. Ajoutez le sensor dans le DAG entre `stocker_hdfs` et `analyser_logs` :

📁 Schéma d'architecture

```
attendre_fichier = HdfsFileSensor(  
    task_id="attendre_fichier_hdfs",  
    hdfs_path="/logs/ecommerce/raw/{{ ds }}.log",  
    namenode_url="{{ var.value.hdfs_url }}",  
    poke_interval=30,      # Vérifie toutes les 30 secondes  
    timeout=600,           # Abandonne après 10 minutes  
    mode="reschedule",     # Libère le worker entre chaque poke (économie de ress  
)
```

3. Testez en deux scénarios :

- **Fichier présent** : le sensor doit terminer immédiatement.
- **Fichier absent** : attendez 2 poking cycles puis uploadez manuellement.

4. Comparez les modes `poke` vs `reschedule` — lequel économise des workers Celery ?

Question : Pourquoi `mode="reschedule"` est recommandé en production avec CeleryExecutor ?

Exercice 2 — Compaction hebdomadaire des logs HDFS (☆☆☆)

Contexte : Après 7 jours d’ingestion, HDFS contient 7 petits fichiers de logs (un par jour). Les “small files” sont un anti-pattern HDFS qui dégrade les performances du NameNode.

Objectif : Créer un **second DAG** `logs_compaction_dag` qui compacte les fichiers journaliers en un fichier hebdomadaire.

ARCHITECTURE

📁 Schéma d'architecture

HDFS avant compaction :

```
/logs/ecommerce/raw/2024-01-01.log  (500 lignes)  
/logs/ecommerce/raw/2024-01-02.log  (500 lignes)  
...  
/logs/ecommerce/raw/2024-01-07.log  (500 lignes)
```

HDFS après compaction :

```
/logs/ecommerce/weekly/2024-W01.log  (3500 lignes)  
/logs/ecommerce/raw/ → vide (fichiers archivés)
```

TRAVAIL DEMANDÉ

Créez `dags/logs_compaction_dag.py` avec les tâches suivantes :


```
@dag(
    dag_id="logs_compaction_dag",
    schedule="0 3 * * 1", # Lundi 3h – compacte la semaine précédente
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=["hdfs", "compaction", "maintenance"],
)

def logs_compaction_dag() -> None:

    @task()
    def lister_fichiers_semaine() -> list[str]:
        """
        Liste les fichiers de la semaine précédente dans HDFS.
        Utilise WebHDFS op=LISTSTATUS.
        Retourne la liste des chemins HDFS.
        """
        # TODO : GET /webhdfs/v1/logs/ecommerce/raw/?op=LISTSTATUS
        # Filtrer les fichiers des 7 derniers jours
        ...

    @task()
    def fusionner_fichiers(fichiers: list[str]) -> str:
        """
        Lit chaque fichier depuis HDFS et les concatène.
        Upload le résultat dans /logs/ecommerce/weekly/YYYY-WXX.log
        Retourne le chemin HDFS du fichier consolidé.
        """
        ...

    @task()
    def valider_compaction(chemin_weekly: str, fichiers_source: list[str]) -> None:
        """
        Vérifie que le fichier weekly a bien plus de lignes que la somme des sources.
        Lève une ValueError si la validation échoue.
        """
        ...

    @task()
    def supprimer_fichiers_journaliers(fichiers: list[str]) -> None:
        """
        Supprime les fichiers journaliers après validation réussie.
        WebHDFS : DELETE /webhdfs/v1{path}?op=DELETE
        """
        ...
```

Implémentez toutes les tâches et chaînez-les correctement.

Bonus : Ajoutez une dépendance inter-DAG avec `ExternalTaskSensor` pour vous assurer que le DAG d'ingestion du dernier jour s'est bien terminé avant de compacter.

Exercice 3 — Retry avec backoff exponentiel et test de résilience (☆☆)

Contexte : L'API HDFS est parfois instable. En production, les tâches doivent réessayer automatiquement avec un délai croissant pour éviter de surcharger le système défaillant.

Objectif : Configurer le **retry exponentiel** et simuler une panne pour tester la résilience.

CONCEPT RETRY EXPONENTIEL

Schéma d'architecture

```
# retry_delay × 2^n → 30s, 60s, 120s, 240s...
default_args = {
    "retries": 4,
    "retry_delay": timedelta(seconds=30),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=10), # Plafonne le délai
}
```

TRAVAIL DEMANDÉ

1. Configurez les retry exponentiels dans `logs_ecommerce_hdfs_dag` :
 - 4 retries max sur `stocker_hdfs`
 - Délai initial 30s, maximum 5 minutes
2. Créez une tâche de **simulation de panne** qui échoue intentionnellement les 2 premières tentatives :

📁 Schéma d'architecture

```
@task(retries=2, retry_delay=timedelta(seconds=10))
def tache_instable(**context) -> str:
    """
    Simule une tâche instable en utilisant le numéro de tentative.
    Échoue les 2 premières fois, réussit à la 3ème.
    """
    tentative = context["ti"].try_number
    log.info("Tentative n°%d", tentative)

    if tentative < 3:
        raise ConnectionError(f"[SIMULÉ] HDFS indisponible – tentative {tentative}")

    log.info("✓ Connexion HDFS rétablie à la tentative %d", tentative)
    return "hdfs://namenode/logs/ecommerce/raw/test.log"
```

3. Déclenchez le DAG et observez dans l'UI :

- L'état de la tâche `tache_instable` passe par : `up_for_retry` → `up_for_retry` → `success`
- Les délais entre tentatives augmentent exponentiellement

4. Analysez les logs : à quel moment exact la tâche passe-t-elle en `up_for_retry` ?

5. **Bonus** : Ajoutez un callback `on_retry_callback` qui loggue le numéro de tentative et le délai avant la prochaine.

Question de réflexion : Si une tâche HDFS doit absolument réussir (SLA critique), quelle combinaison `retries` + `retry_delay` + `execution_timeout` choisiriez-vous et pourquoi ?

Exercices supplémentaires — Jour 2 — Formation Airflow IPSSI Montpellier